Week 4 Project Progress Report

Adaptive Fabric Surface Defect Detection and Localization using Classical Texture Descriptors

Course Name: ECE501 Digital Image Processing (DIP)
Group Number: Group 15
Group Members: Rinal Shah, Varun Hotani, Aadi Mehta, Pratik Agrawal
Week Number: Week 4
Date: October 18, 2026
Deliverable: Production-grade refactoring of the detection pipeline (packaging, correctness fixes, configuration, tests, CI)
Branch: `refactor/production-grade` (20 commits on top of the mid-semester report, including the merge of the Week-5 accuracy branch)

Objectives of the Week

- Turn the flat `Codes/` script collection into an installable, importable Python package with one clear responsibility per module, while keeping the `Codes/` folder required by the course submission rules.
- Fold the unmerged accuracy work (fabric ROI cropping, illumination flattening, robust hybrid scoring) into the main line so that every later experiment starts from one code base.
- Fix the data-handling defects listed as open evidence issues in the mid-semester notes: wrong AFID defect-code map, invented fabric names, swapped strip orientation, silent empty ground truth, silent synthetic fallback.
- Remove two methodological weaknesses: an in-sample threshold calibration and the reporting of undefined metrics as 0.
- Make every run reproducible and auditable: one typed configuration, a command-line interface, structured logging, and a machine-readable record of each run.
- Add automated tests and continuous integration so that these fixes stay fixed.

Summary of the Week

Item	Outcome
Commits	20 single-purpose commits (1 merge, 6 fixes, 6 refactors, 2 features, 1 performance, 1 test, 1 CI, 2 chores)
Package	`Codes/fabric_defect/`: 13 modules, 1,645 lines; installable with `pip install -e "Codes[dev]"`
Correctness	Defect labels of all 13 mid-semester test images were wrong and are now corrected; strip orientation, mask handling, synthetic fallback and a pandas-3 crash fixed
Methodology	Threshold τ now calibrated on held-out defect-free images; undefined ratios reported as NaN
Speed	Patch scoring vectorised: 17–20× faster for $\geq$ 1,000 patches, bit-identical scores
Quality gates	42 automated tests (unit + end-to-end), GitHub Actions on Python 3.11 and 3.13
Reproducibility	Typed config + TOML file, CLI, logging, `run_config.json` with git SHA and package versions

The AFID images were not available on the machine used this week, so no new AFID benchmark numbers are reported. All AFID numbers below are re-analyses of the per-image counts already committed from the mid-semester run; all other numbers come from unit tests or from the synthetic smoke benchmark, and are labelled as such.

Work Completed This Week

A. Repository and Package Restructure

Until Week 3 the code was ten loose files in `Codes/`. Every file imported its siblings as `from Codes.x import ...`, which only worked because `run_pipeline.py` inserted the repository root into `sys.path` at import time. Configuration was a set of module-level constants, plotting lived inside the pipeline runner, and there was no test or entry point other than running that one script.

The course submission rules fix the top-level layout (`Codes/`, `Results/`, `Mid_Sem_Report/`, `End_Sem_Report/`), so `Codes/` is kept as the code root and a proper package is placed inside it. Figure 1 shows the before/after tree and Table 1 lists what each module now owns.

Before (Week 3 / mid-sem)

```
Codes/
  __init__.py
  config.py # constants
  dataset.py # AFID + synthetic
  evaluation.py
  features_glcm.py
  features_lbp.py
  preprocessing.py
  reference_model.py
  run_pipeline.py # sys.path hack,
                  # plotting, main
  segmenter.py
requirements.txt # 9 deps
```

After (Week 4)

```
Codes/
  pyproject.toml # installable package
  configs/default.toml # every tunable
  fabric_defect/
    __main__.py cli.py # python -m fabric_defect
    config.py # typed dataclasses
    afid.py # AFID catalog + masks
    synthetic.py # dev/test data only
    preprocessing.py features.py
    model.py segmentation.py
    metrics.py pipeline.py viz.py
  tests/ # 42 tests
.github/workflows/tests.yml
End_Sem_Report/.gitkeep
```

Figure 1: Repository layout before and after the refactor. The `Results/`, `Mid_Sem_Report/` and `reports/` folders are unchanged.

Table 1: Module responsibilities in `Codes/fabric_defect/`.

Module	Lines	Responsibility
`config.py`	144	Frozen dataclass tree for all tunables, validation, TOML loading
`afid.py`	118	AFID catalog for the published layout, official defect codes, filename parsing, multi-part mask loading
`synthetic.py`	229	Synthetic 256×4096 strips with injected defects (smoke tests only)
`preprocessing.py`	153	Fabric ROI, illumination flattening, median/Gaussian filtering, CLAHE, patch extraction
`features.py`	165	GLCM (30-D) and multi-scale LBP (28-D) extractors, fused descriptor
`model.py`	193	Per-fabric reference model, vectorised Mahalanobis, MAD/hybrid scoring, held-out τ, deviation map
`segmentation.py`	100	Thresholding and morphological clean-up
`metrics.py`	138	Patch and pixel metrics (NaN convention), per-fabric summaries
`pipeline.py`	253	Orchestration: fit $\rightarrow$ calibrate $\rightarrow$ evaluate $\rightarrow$ write results and run record
`viz.py`	72	Inspection collage and summary charts (headless backend)
`cli.py`, `__main__.py`	74	Command-line interface and exit codes

Figure 2 shows the dependency structure. Dependencies point one way only: the leaf modules

(preprocessing, features, model, segmentation, metrics) import nothing from the package except `config`, so each can be unit-tested in isolation. Only `pipeline.py` knows the order of the stages.

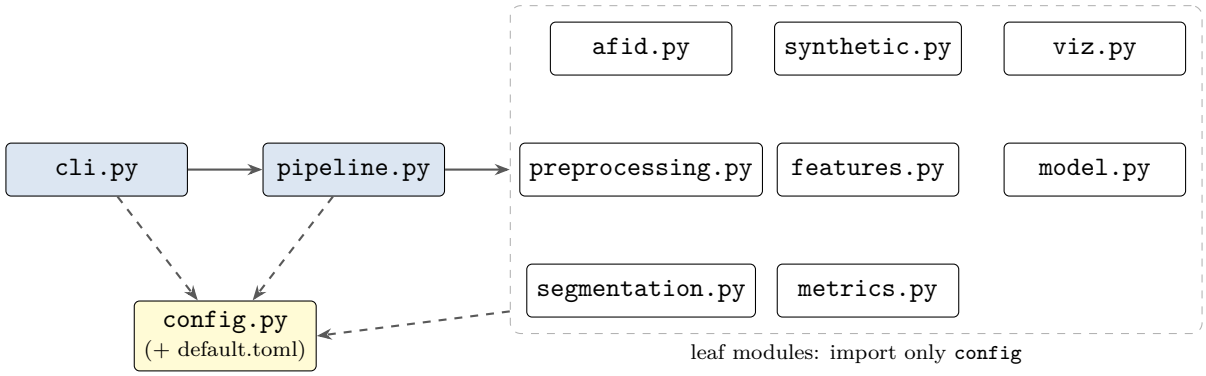


Figure 2: Module dependencies. Solid arrows: run-time calls from the orchestrator into the leaf modules. Dashed arrows: configuration. No leaf module imports another leaf module.

The package is declared in `Codes/pyproject.toml`, which replaces the `sys.path` hack with a normal editable install. Three dependencies that nothing imported (`reportlab`, `PyPDF2`, `scikit-learn`) were dropped, and `scipy` is no longer imported directly once scoring was vectorised (it is still installed by `scikit-image`). `opencv-python` became `opencv-python-headless`, because the pipeline never opens a window and the headless wheel installs on CI runners without GUI libraries.

B. Integration of the Week-5 Accuracy Branch

Pratik's unmerged branch from 11 October (`origin/week-5`) carried the accuracy work behind the mid-semester numbers. It was merged first (commit `fd47eab`), so that the refactor applies to the code that actually produced the reported results. It contributed four techniques, which are now configurable:

Fabric ROI. Uncropped conveyor background and frayed selvage columns are removed before patching. A column c is fabric if its mean intensity is below 245 and its standard deviation above 2. The ROI is the span of fabric columns, shrunk by a margin $m = 40\,\text{px}$ on each side; it falls back to the full width if fewer than one patch width remains.

Illumination flattening. Slow lighting gradients and vignetting are removed by subtracting a heavily blurred background:

$$I_{\text{flat}}(x,y) = \text{sat}_{[0,255]}\Big(I(x,y) - (G_\sigma * I)(x,y) + 128\Big), \qquad \sigma = 50 \text{ px}, \tag{1}$$

which acts as a high-pass filter with a cut-off far below the yarn frequency (a period of 4–8 px), so texture is kept and shading is removed.

Adaptive GLCM quantisation. Each patch is min–max stretched before quantisation to $L = 64$ levels when its range exceeds 5 grey levels, so low-contrast fabrics still use the full co-occurrence matrix.

Robust hybrid scoring. Each strip is also compared with *itself* using median and MAD statistics, which tolerate a minority of defective patches [2, 3]. For the patch descriptors $\mathbf{x}_i$ of one strip:

$$r_i = \Big\| (\mathbf{x}_i - \operatorname{med}_j \mathbf{x}_j) \oslash \max\big(\operatorname{MAD}_j \mathbf{x}_j,\, 10^{-4}\big) \Big\|_2, \tag{2}$$

$$z_i^{\text{rel}} = \max\Big(0,\ \frac{r_i - \operatorname{med}_j r_j}{\operatorname{MAD}_j r_j + 10^{-4}}\Big), \qquad z_i^{\text{proto}} = \max\Big(0,\ \frac{d_f(\mathbf{x}_i) - \operatorname{med}_j d_f(\mathbf{x}_j)}{\operatorname{MAD}_j d_f(\mathbf{x}_j) + 10^{-4}}\Big), \tag{3}$$

$$s_i = 0.6\, z_i^{\text{rel}} + 0.4\, z_i^{\text{proto}}, \qquad \text{patch flagged if } s_i \geq k = 2.5, \tag{4}$$

where $\oslash$ is element-wise division and d_f is the Mahalanobis distance to the fabric-f reference (Section E). The scoring mode (`hybrid`, `relative` or `mahalanobis`) is now a config value instead of a code edit.

C. Data-Handling Correctness Fixes

These are the most consequential changes of the week. Several of them silently changed what the evaluation measured.

C1. Official AFID defect codes. AFID filenames have the form `nnnn_ddd_ff.png`: image id, defect code, fabric code [1]. The old map in `dataset.py` numbered the defect classes 001–012 in list order, and the Week-5 branch extended it with names that do not occur in AFID (e.g. `oil_stain`, `broken_warp_end`). The new map uses the 12 official codes (Table 2).

Table 2: AFID defect codes: official name (AFID paper [1]) vs. the label produced by the Week-5/mid-semester code.

Code	Official name (now used)	Old label	
002	broken end	broken_pick	wrong
006	broken yarn	cut_selvage	wrong
010	broken pick	contamination	wrong
016	weft curling	warp_break	wrong
019	fuzzy ball	broken_warp_end	wrong
022	cut selvage	snarl	wrong
023	crease	coarse_pick	wrong
025	warp ball	loose_warp	wrong
027	knot	oil_stain	wrong
029	contamination	double_pick	wrong
030	nep	weft_crack	wrong
036	weft crack	hole	wrong
000	(defect-free)	—	

Re-parsing the 13 images of the committed mid-semester run (`Results/metrics_summary.csv`) shows that **all 13 defect labels were wrong** (Table 3). Every per-defect statement in earlier reports is therefore unreliable and must be redone; per-image and overall metrics are not affected, because the labels were only used for grouping. The Week-5 inspection figure for `0001_002_00` (Figure 3) agrees with the correction: the figure title says "broken_pick", but the ground-truth mask is a single thin line along the 256-row (fabric travel) axis, which is a warp-direction fault, i.e. a broken end.

C2. Fabric identity. The fabric used to be derived as `int(ff) \% 7` and mapped onto the names Plain, Twill, Satin, Rib, Basket, Jacquard and Herringbone. AFID does not publish weave names for its fabric codes, so these names were invented. Fabrics are now reported as `fabric_00` … `fabric_06` straight from the filename. The weave names survive only inside `synthetic.py`, where they describe the synthetic textures.

Table 3: Labels of the 13 mid-semester test images before and after the fix (re-parsed from committed filenames; no re-run needed).

Image	Old fabric	Old defect	New fabric	New defect
0001_002_00, 0002_002_00	Fabric01_Plain	broken_pick	fabric_00	broken_end
0004_002_01, 0005_002_01	Fabric02_Twill	broken_pick	fabric_01	broken_end
0010_006_02, 0011_006_02	Fabric03_Satin	cut_selvage	fabric_02	broken_yarn
0018_010_03	Fabric04_Rib	contamination	fabric_03	broken_pick
0019_016_03	Fabric04_Rib	warp_break	fabric_03	weft_curling
0043_019_04, 0044_019_04	Fabric05_Basket	broken_warp_end	fabric_04	fuzzy_ball
0094_027_05	Fabric06_Jacquard	oil_stain	fabric_05	knot
0055_019_06, 0056_019_06	Fabric07_Herringbone	broken_warp_end	fabric_06	fuzzy_ball

13 of 13 defect labels changed; fabric grouping unchanged (Fabric$k$ was code $k-1$ by index).

C3. Strip orientation. AFID strips are 256 rows (fabric travel) by 4096 columns (fabric width). The config declared `STRIP_HEIGHT=4096, STRIP_WIDTH=256`. As a result, the synthetic generator drew strips the wrong way round, and the inspection figure laid five tall panels side by side, squeezing the 4096-px axis into about a fifth of the page (Figure 3). The geometry is corrected, the synthetic generator now places defects along the long axis, and the figure stacks the five panels vertically (Figure 4).

C4. Ground-truth masks. Two silent failure modes were removed:

1. *Missing mask → all-zero ground truth.* When no mask file was found, the old runner substituted `np.zeros_like(image)`. Every detection on that strip then counted as a false positive and every recall became 0/0, reported as 0. Strips without a mask are now skipped with a logged warning.
2. *Multi-part masks.* The loader looked only for `<stem>_mask.png`. The catalog now collects every `<stem>_mask*.png` and `afid.load_mask` merges them with a logical OR, resizing with nearest-neighbour interpolation if needed, so a defect delivered as several mask files is no longer partly dropped.

A strip whose merged mask is empty (e.g. `0044_019_04`, which has 0 ground-truth pixels in the mid-semester run) is still evaluated, but it is logged and its recall-type metrics become NaN (Section G).

C5. No silent synthetic fallback. If `data/raw` was empty, the old runner generated synthetic strips into that folder and then benchmarked them under the heading "AITEX AFID". This is how the Week-3 results table came to be computed on synthetic data. Generation now lives in `synthetic.py` and runs only with the explicit `-synthetic` flag; its output is logged and titled "SYNTHETIC". A missing AFID folder now stops the run with exit code 2 and a message saying which sub-folders are missing. The tracked `data/raw/aitex_manifest.csv`, a catalog of synthetic files with Windows paths, was removed. While moving the generator, a reproducibility bug was fixed as well: the per-defect seed used `hash(defect_name)`, and Python salts string hashes per process, so the "reproducible" synthetic benchmark changed on every run. It now uses the defect's index, and the injected noise comes from a seeded generator instead of the global `np.random`.

C6. pandas 3 crash. Stratified test sampling used `groupby(...).apply(lambda g: g.head(n))`. On pandas $\geq$ 3 this drops the grouping column, so the next line failed with `KeyError: 'fabric_type'`. The equivalent built-in `groupby(...).head(n)` keeps the column and is simpler.

D. Typed Configuration and Reproducibility

All 24 tunables now live in one frozen dataclass tree (`PipelineConfig`, with sections `preprocess`, `patch`, `glcm`, `lbp`, `model`, `segment`). Constructors take their section instead of reading globals, which is what makes the leaf modules testable with non-default settings. The values are validated when constructed: for example, an even median kernel, a stride larger than the patch, an unknown scoring mode or a calibration fraction outside $[0, 1)$ raises immediately rather than failing deep in OpenCV. A TOML file may override any subset of keys; unknown keys are rejected so that typos cannot be silently ignored. The committed `Codes/configs/default.toml` is tested to equal the dataclass defaults exactly. Table 4 lists the defaults; they reproduce the mid-semester settings, and a regression run confirmed that this commit left every per-image metric unchanged.

Table 4: Default configuration (`Codes/configs/default.toml`).

Section	Parameters
`preprocess`	median 3 px; Gaussian off; CLAHE [12] clip 2.0, grid 8×8; illumination $\sigma = 50$; selvage margin 40 px
`patch`	size 256, stride 128 (50 % overlap); patch labelled defective if > 25 mask pixels
`glcm` [10]	distances $\{1, 2, 3\}$; angles $\{0°, 45°, 90°, 135°\}$; 64 levels; contrast, dissimilarity, homogeneity, energy, correlation (mean and std over angles) $\rightarrow$ 30-D
`lbp` [11]	uniform riu2 at $(P, R) = (8, 1)$ and $(16, 2) \rightarrow 10 + 18 = 28$-D
`model`	scoring `hybrid`; $\lambda = 10^{-4}$; $k = 2.5$; calibration fraction 0.25
`segment`	5×5 rectangular opening/closing; components < 50 px removed
run	14 test strips (stratified by fabric); seed 0

Every run writes `run_config.json` next to its CSVs. It records the start time (UTC), the git commit SHA and a dirty-tree flag, the data folder, whether the run was synthetic, the Python and package versions, and the fully resolved configuration. Any number in a future report can therefore be traced to the exact code and settings that produced it, which closes the provenance gaps noted in the mid-semester notes [9, 8].

E. Vectorised Mahalanobis Scoring

The anomaly score of a patch descriptor $\mathbf{x} \in \mathbb{R}^{58}$ against fabric f is

$$d_f(\mathbf{x}) = \sqrt{(\mathbf{x} - \boldsymbol{\mu}_f)^\top \, \boldsymbol{\Sigma}_{f,\mathrm{reg}}^{+} \, (\mathbf{x} - \boldsymbol{\mu}_f)}, \qquad \boldsymbol{\Sigma}_{f,\mathrm{reg}} = \boldsymbol{\Sigma}_f + \lambda \, \tfrac{\mathrm{tr}(\boldsymbol{\Sigma}_f)}{D} \, \mathbf{I}, \tag{5}$$

with the Moore–Penrose pseudo-inverse $^+$ and trace-scaled Tikhonov regularisation [5, 6]. The old code evaluated this with a Python loop over patches. For a batch $\boldsymbol{\Delta} = \mathbf{X} - \mathbf{1}\boldsymbol{\mu}_f^\top \in \mathbb{R}^{N \times D}$ all squared distances are the row sums of $(\boldsymbol{\Delta}\boldsymbol{\Sigma}^{+}) \odot \boldsymbol{\Delta}$, now computed with a single `np.einsum("ij,jk,ik->i", ...)`. The pseudo-inverse fallback now catches only `LinAlgError` instead of every exception. Table 5 gives timings (best of 7 runs, 58-D features, reference fitted on 2,000 samples); the scores agree with the loop to the last bit.

Table 5: Scoring time per batch: per-patch loop (before) vs. vectorised (after).

Patches N	Loop (ms)	Vectorised (ms)	Speed-up
31 (one AFID strip)	0.105	0.034	3.1×
1,000	3.945	0.203	19.5×
20,000	93.605	5.409	17.3×
Max. absolute difference between the two implementations: 0.			

To keep this in proportion we also profiled one 256×4096 strip end to end (31 patches): preprocessing 267 ms, GLCM 100 ms, **LBP 577 ms**, scoring 0.2 ms. Scoring was never the bottleneck.

The real cost is LBP, which is recomputed for every overlapping patch (each pixel twice at stride 128), and the $\sigma = 50$ Gaussian of Eq. (1). Both are targets for next week (see Plan).

F. Held-Out Threshold Calibration

In the mahalanobis mode the threshold is $\tau_f = \mu_{d,f} + k\,\sigma_{d,f}$, where $\mu_{d,f}, \sigma_{d,f}$ are the mean and standard deviation of d_f over defect-free patches. Previously these were computed on the *same* patches used to fit $\boldsymbol{\mu}_f$ and $\boldsymbol{\Sigma}_f$. In-sample distances are systematically too small, more so when N is not much larger than $D = 58$ [4], so τ_f came out too low and normal texture on unseen strips crossed it too often.

The defect-free images of each fabric are now split *by image*, with a seeded permutation, into a fit set and a calibration set (25 % by default, at least one image on each side whenever there are ≥ 2 images):

$$\boldsymbol{\mu}_f, \boldsymbol{\Sigma}_f \leftarrow \mathcal{X}_f^{\text{fit}}, \qquad \mu_{d,f}, \sigma_{d,f} \leftarrow \{\, d_f(\mathbf{x}) : \mathbf{x} \in \mathcal{X}_f^{\text{cal}} \,\}. \tag{6}$$

Splitting by image rather than by patch matters: the 50 % overlapping patches of one strip are strongly correlated, so a patch-level split would leak the fit data into the calibration data. The model records whether τ was calibrated out of sample, and the log says so per fabric.

Table 6 tests the effect on synthetic data, the only data available this week. Per fabric, the reference is fitted on 3 strips (93 patches), τ is set either in-sample or on a 4th strip, and the false-positive rate is measured on a 5th, unseen defect-free strip. Held-out calibration raises τ in 6 of 7 fabrics and halves the mean false-positive rate (6.45 % → 3.23 %). With one calibration strip the estimate is noisy, as fabric 3 shows, which is why AFID (about 20 defect-free images per fabric) is needed for the real comparison.

Table 6: In-sample vs. held-out τ (**synthetic data**, mahalanobis mode, $k = 2.5$). FPR is measured on 31 patches of an unseen defect-free strip.

Synthetic fabric	Fit patches	τ in-sample	τ held-out	FPR in-sample (%)	FPR held-out (%)
Fabric01_Plain	93	3.68	4.18	3.2	0.0
Fabric02_Twill	93	3.88	4.01	3.2	3.2
Fabric03_Satin	93	3.77	3.49	3.2	9.7
Fabric04_Rib	93	4.26	4.86	16.1	0.0
Fabric05_Basket	93	3.59	3.71	9.7	6.5
Fabric06_Jacquard	93	4.31	5.18	9.7	3.2
Fabric07_Herringbone	93	3.92	4.08	0.0	0.0
Mean		**3.92**	**4.22**	**6.45**	**3.23**

G. Metric Conventions: NaN Instead of Zero

Every ratio whose denominator was zero used to be reported as 0. In particular, an image with no ground-truth defect pixels got recall 0, which reads as "missed everything" when there was nothing to find. Averaged into a mean, these zeros pulled results down and made "undefined" and "failed" look the same, which was the open question about fabric codes 04 and 06 in the mid-semester notes. The new rules are in Table 7. F1 and Dice use the count form $2TP/(2TP + FP + FN)$, so a real miss (positives present, none found) still scores 0, and only "no positives and no predictions" is undefined. Summaries use NaN-skipping means and report n, the number of images behind each mean.

Applied to the committed per-image counts of the mid-semester run (TP/FP/TN/FN and pixel counts are stored, so no images are needed), two means change (Table 8). Image `0044_019_04` has an empty mask, and `0055_019_06` has no defective patch under the > 25-pixel rule (its mask has only 22 pixels). Both had been counted as complete misses. The zeros of fabric codes 04 and 06 are

Table 7: Undefined-metric rules (implemented in `metrics.py`, enforced by tests).

Metric	Undefined when	Reported as
Precision (patch / pixel)	nothing predicted defective	NaN (was 0)
Recall (patch / pixel)	nothing defective in ground truth	NaN (was 0)
F1 / Dice	no ground-truth positives *and* no predictions	NaN (was 0; pixel case was 1)
IoU (pixel)	prediction and ground truth both empty	NaN (was 1)
Specificity	no ground-truth negatives	NaN (was 0)

therefore partly "nothing to detect", not only missed detections. All other zeros in those rows are real misses, since their denominators are positive.

Table 8: Mid-semester 13-image run re-scored under the NaN convention (mean of per-image values; AFID data, no re-run).

Metric	Old mean (%), $n = 13$	New mean (%)	n	Undefined images
Patch precision	21.88	21.88	13	—
Patch recall	41.54	**49.09**	11	0044_019_04, 0055_019_06
Patch F1	27.13	27.13	13	—
Pixel recall	43.57	**47.20**	12	0044_019_04
Pixel precision	0.36	0.36	13	—
Pixel IoU	0.36	0.36	13	—

H. Command-Line Interface, Logging and Figures

The pipeline now runs through one entry point, available as `python -m fabric_defect` or as the installed `fabric-defect` command:

```
pip install -e "Codes[dev]"
python -m fabric_defect run # AFID in data/raw -> Results/
python -m fabric_defect run --config my.toml --out Results/runs/exp1
python -m fabric_defect run --scoring mahalanobis --seed 1 --max-test 30
python -m fabric_defect run --synthetic --out /tmp/smoke # smoke test, not AFID
```

Exit codes are meaningful for scripting: 0 for success, 1 if no strip could be evaluated, 2 for a missing dataset or an invalid config. The heavy imports (OpenCV, scikit-image, matplotlib) are deferred, so `-help` answers instantly.

All `print` calls were replaced by the standard `logging` module, with timestamps, levels and module names and a `-log-level` switch. Every place that used to skip a strip or fabric silently now logs a warning saying why: no reference model, no mask, empty mask, or ROI narrower than one patch.

Plotting moved to `viz.py`. It uses the non-interactive `Agg` backend, so it runs on CI and servers, and it uses the figure API rather than global `plt` state. Figures 3 and 4 compare the old and new inspection layouts.

Testing and Continuous Integration

The repository had no tests before this week. `Codes/tests/` now holds 42 `pytest` tests (Table 9) that run in about 30 s on a laptop. Each test pins down a behaviour that was wrong before or that the refactor must not change. GitHub Actions (`.github/workflows/tests.yml`) installs the package and runs the suite on every push and pull request, on Python 3.11 (the oldest supported, because `tomllib` is used) and Python 3.13.

Writing the tests exposed one further bug. `GLCMFeatureExtractor.get_feature_names()` listed names as mean/std *interleaved* per distance, while the feature vector stores all means first

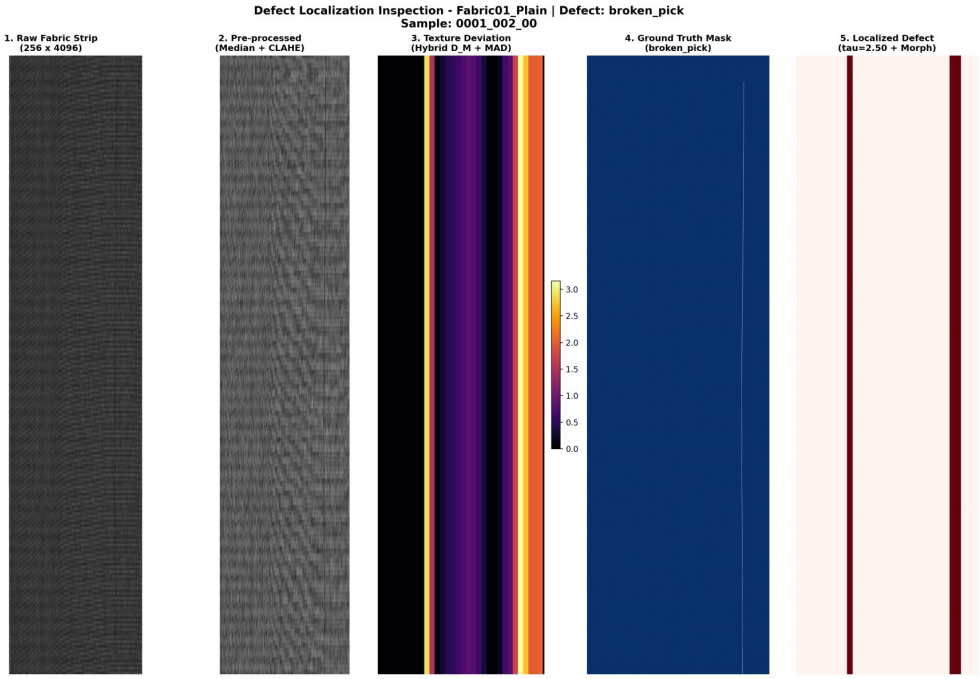


Figure 3: Before (AFID image `0001_002_00`, produced by the Week-5/mid-semester code). The 256×4096 strip is drawn as five side-by-side tall panels, which compresses the long axis. The title and mask label say "broken_pick", but code 002 is officially a broken end, and the single warp-direction line in the ground-truth panel agrees with the official name.

and then all stds for each property. Any feature-importance plot built from these names would have attached 20 of the 30 GLCM names to the wrong value. It was fixed when the extractors were merged into `features.py`.

Commit History

All commits are on `refactor/production-grade`, each doing one thing so it can be reviewed or reverted on its own (Table 10).

Table 10: Commits of the week, oldest first.

Hash	Message
`fd47eab`	Merge origin/week-5 into refactor/production-grade
`1c015fc`	chore: add pyproject and drop unused dependencies
`31f6daa`	refactor: move modules into the fabric_defect package
`f508ebb`	refactor: merge glcm and lbp extractors into features module
`f4097b4`	fix: use official AFID defect codes and fabric ids from filenames
`2dcdf5d`	fix: correct AFID strip orientation to 256x4096
`04c9b12`	fix: merge multi-part masks and skip images without masks
`1346049`	refactor: split synthetic generator out of the dataset loader
`84bcd5d`	fix: keep fabric column when stratifying test strips
`a29c922`	refactor: replace config globals with typed dataclasses
`ee92af5`	perf: vectorise Mahalanobis scoring
`ea13152`	fix: calibrate threshold on held-out defect-free images
`0d6a950`	fix: report undefined metrics as NaN instead of zero
`2b0df7f`	refactor: move plotting into viz module
`aac762a`	refactor: replace print calls with logging
`9983eaa`	feat: add command line interface

Hash	Message
`2c212c1`	feat: write run config and git sha alongside results
`8792876`	test: add unit tests and synthetic smoke test
`b5bd696`	ci: run pytest on push and pull request
`e591877`	chore: add End_Sem_Report placeholder

Literature Review

Topic / Paper	Key Takeaway	Relevance to the Project
A public fabric database for defect detection methods and results [1]	Defines AFID: 245 images of 256 × 4096 px, 7 fabrics, 12 defect classes with numeric codes, pixel masks, and the `nnnn_ddd_ff` naming scheme.	Authoritative source for the corrected code table (Table 2) and the corrected strip orientation.
Alternatives to the median absolute deviation [2]	MAD-based scale estimates have a 50 % breakdown point, while the standard deviation breaks down with a single outlier.	Justifies the median/MAD normalisation of the hybrid score (Eqs. 2–4): defective patches cannot inflate the strip's own scale estimate.
Detecting outliers: use absolute deviation around the median [3]	Practical case for median ± k·MAD cut-offs instead of mean ± k·SD, with guidance on choosing k.	Supports a fixed robust cut-off $k = 2.5$ on the robust z-scores of each strip.
The Elements of Statistical Learning, Ch. 7 [4]	Training-set error is optimistically biased; model assessment needs data not used for fitting.	Motivates calibrating τ on held-out defect-free images (Eq. 6, Table 6).
On the generalized distance in statistics [5]; Solution of incorrectly formulated problems [6]	Covariance-aware distance; ridge regularisation of ill-posed inverses.	Eq. (5), now evaluated in vectorised form with identical results.
Hidden technical debt in machine learning systems [7]	Most of a real system is glue code, configuration and data handling, and these are where silent failures accumulate.	The bugs found this week (wrong labels, silent fallbacks, untracked config) are exactly this kind of debt; typed config and fail-loud data loading address it.
Best practices for scientific computing [8]	Write tests, automate, keep provenance, use version control with small changes.	Basis of the test suite, CI and single-purpose commits.
Improving reproducibility in ML research [9]	Report exact code version, settings, data splits and seeds with every result.	`run_config.json` stores git SHA, resolved config, seed and package versions per run.

Quantitative Results and Analysis

Code Metrics

The package has almost the same size as before (+26 lines), even though it now includes a CLI, run provenance, config validation and the new mask handling. Duplicate code was removed (e.g. the two dataset layouts, the two extractor call sites) and roughly the same amount was spent on checks and messages.

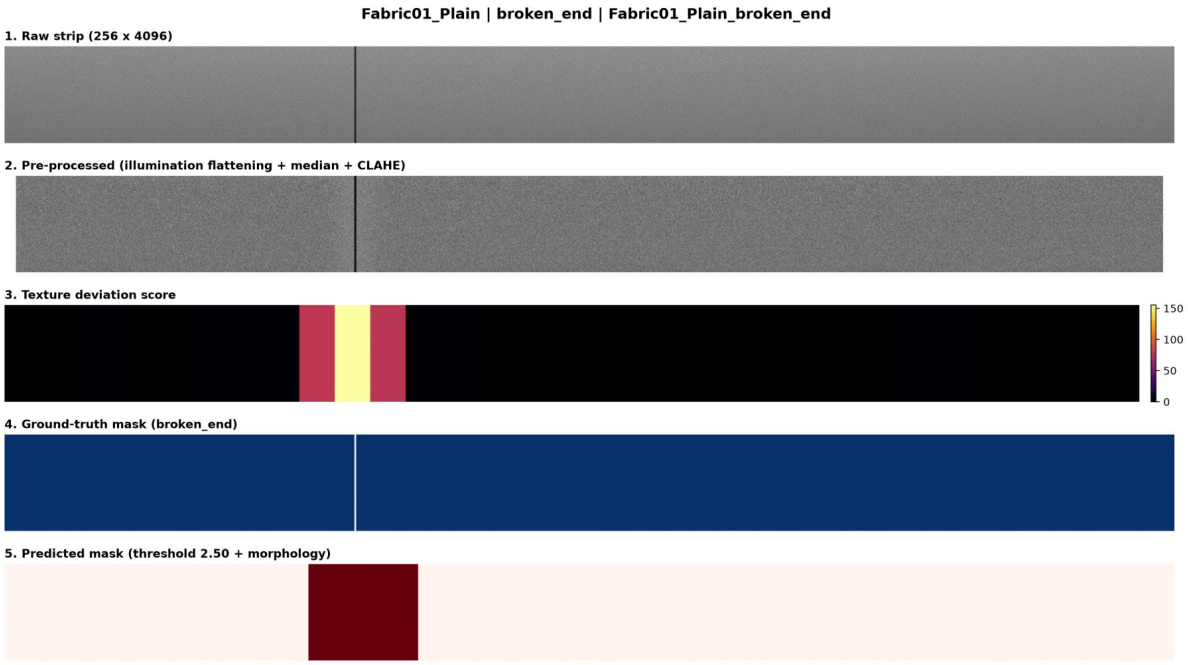


Figure 4: After (**synthetic** strip, smoke run of the refactored package). The panels are stacked along the 4096-px axis: raw, pre-processed, hybrid deviation score, ground truth, prediction. The patch-level detection is correctly placed, and its width (one to two 256-px patches around a 6-px defect) shows the coarse-localization limit discussed below.

Synthetic Smoke Benchmark

Table 13 is the output of `python -m fabric_defect run -synthetic` with default settings (14 strips, 2 per synthetic fabric). **These are not AFID results.** They show that the full pipeline runs end to end on the new orientation, and they expose one behaviour of the hybrid score worth knowing before the AFID re-run.

Recall is about 50 % for every fabric because the two strips per fabric are a broken end (a 6-px column, Figure 4) and a broken pick. The broken end is found, while the broken pick (a 6-px row running across the full 4096-px width) is missed entirely. A full-width defect sits in *every* patch, so the within-strip median absorbs it and the relative term z^{rel} of Eq. (3) is zero everywhere. Only the prototype term can still see it, and it is itself median-normalised per strip. This blind spot is inherent to scoring a strip against itself, and it matters for AFID, where broken picks (code 010) and weft-direction faults span the fabric width. The `mahalanobis` mode does not have this blind spot and is now one flag away (`-scoring mahalanobis`).

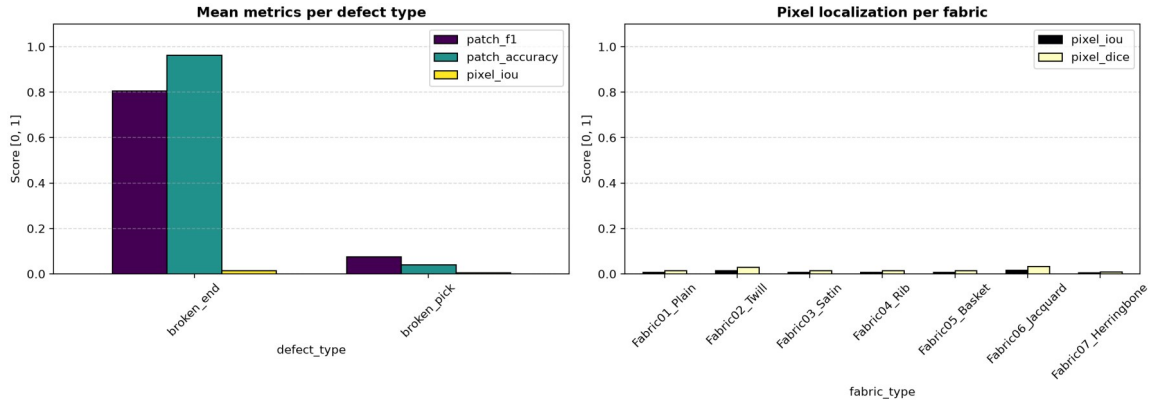


Figure 5: Summary charts written by `viz.py` for the synthetic smoke run (**synthetic data**).

Table 9: Test suite (`Codes/tests/`).

File	Tests	What it guards
`test_afid.py`	14	Official defect codes for 7 real filenames; non-AFID names rejected; multi-part masks OR-merged and resized; empty mask list rejected; missing dataset raises with a hint; catalog on a miniature AFID tree (nested defect-free folders, an image without a mask)
`test_preprocessing.py`	7	Last window reaches the right edge; patch labels use the strict $>$ threshold; ROI offsets; strips smaller than a patch; ROI trims white margins and selvage, with fallback; pre-processor keeps shape and dtype; image-level, seeded, disjoint split
`test_features.py`	5	Descriptor is 58-D and matches its 58 names; LBP histograms sum to 1 per scale; GLCM name order matches the feature order (this was a real bug); flat patches; determinism
`test_model.py`	8	Vectorised score equals Eq. (5); scale invariance of Mahalanobis; errors before fit or on empty data; held-out calibration is used, flagged and raises τ; hybrid score finds an injected outlier; overlap averaging in the deviation map; config validation
`test_metrics.py`	4	Known confusion counts; NaN for undefined precision/recall/F1 while a real miss stays 0; pixel IoU/Dice incl. empty masks; NaN-skipping summary with n
`test_pipeline_smoke.py`	4	`default.toml` equals the defaults; unknown config keys rejected; CLI exit code 2 without data; full synthetic run via the CLI writes metrics, summary, chart and `run_config.json`
Total	**42**	all passing at commit `e591877`

Table 12: Code base before (`fd47eab`, after the Week-5 merge) and after the refactor (`e591877`).

	Before	After
Python modules (excl. tests)	10	13
Lines of package code	1,619	1,645
Lines of test code	0	322
Automated tests	0	42
Declared runtime dependencies	9	5
Configuration	23 module constants	typed dataclasses + TOML
`sys.path` manipulation	yes	no
Silent fall-backs (synthetic data / empty GT)	2	0
CI	none	GitHub Actions, 2 Python versions

Discussion of Findings

1. **Defect-wise conclusions so far are invalid.** Every defect label in the mid-semester 13-image run was wrong (Table 3). For example, the "broken pick" rows were broken ends, and the four "broken warp end" rows were fuzzy balls. The per-defect table in the mid-semester notes already used the official names. Anything grouped by the code's `defect_type` column, such as the Week-3 bar charts, should not be used. Per-image and overall numbers stand.

2. **Some reported zeros were undefined, not failures.** Under the NaN rule, mean patch recall on the mid-semester set is 49.09 % over 11 images (not 41.54 % over 13), and pixel recall is 47.20 % over 12 (not 43.57 % over 13). Precision and IoU are unchanged: localization remains very weak (0.36 % IoU) and that conclusion does not move.

3. **The thresholds were optimistic.** In-sample calibration underestimates the spread of normal scores. On synthetic data, held-out calibration halved the false-positive rate on unseen defect-free strips. This must be confirmed on AFID, where enough defect-free images exist

Table 13: Synthetic smoke run (default config, hybrid scoring; per-fabric means over $n = 2$ strips). **Synthetic data, not AFID.**

Synthetic fabric	Patch acc. (%)	Patch prec. (%)	Patch rec. (%)	Pixel rec. (%)	Pixel IoU (%)
Fabric01_Plain	50.0	100.0	50.0	50.0	0.78
Fabric02_Twill	50.0	70.0	54.8	54.7	1.54
Fabric03_Satin	48.4	66.7	50.0	50.0	0.78
Fabric04_Rib	50.0	100.0	50.0	50.0	0.78
Fabric05_Basket	50.0	83.3	51.6	50.0	0.78
Fabric06_Jacquard	53.2	83.3	54.8	53.7	1.69
Fabric07_Herringbone	50.0	75.0	53.2	50.0	0.47
Overall (14 strips)	**50.2**				**0.97**

per fabric.

4. **Within-strip scoring has a blind spot** for defects spanning the full strip width, a property of the hybrid score rather than of the refactor. A controlled `hybrid` vs. `mahalanobis` comparison on AFID is now a config change, not a code change.
5. **Performance work should target LBP, not scoring.** After vectorisation, scoring takes 0.2 ms per strip versus 577 ms for LBP and 267 ms for pre-processing.
6. **Earlier synthetic results must not be read as AFID results.** The silent synthetic fallback explains how the Week-3 results table (95 % patch accuracy) could be produced without AFID present. With the fallback removed, that cannot happen again unnoticed: a run without data now stops, and synthetic runs are labelled in the logs, the summary line and `run_config.json`.

Challenges Faced

- **No AFID images on the development machine.** Only the (synthetic) manifest was in the repository. Everything was therefore verified with unit tests, the synthetic smoke run, and re-analysis of the committed per-image AFID counts. The one regression check that needs no data, identical per-image metrics before and after the config refactor, was run on the synthetic benchmark.
- **Environment breakage.** The system Python had OpenCV compiled against NumPy 1.x with NumPy 2 installed, so `import cv2` failed. Development moved to a project virtual environment (`.venv`, git-ignored) with the headless OpenCV wheel, which is also what CI uses.
- **Behaviour preservation.** Refactoring and bug fixing are easy to mix up. Pure moves and refactors were kept in separate commits from behaviour changes. The config refactor was checked to reproduce the previous run's per-image metrics exactly, and the vectorised scorer to reproduce the loop's scores bit for bit.
- **Library drift.** pandas 3 changed `groupby().apply` semantics and broke the runner without any code change on our side. Pinning lower bounds and testing on two Python versions in CI is the guard against this.

Plan for Next Week (Week 5)

- **Re-run the AFID benchmark on the fixed code** with `run_config.json` committed alongside, and replace every mid-semester number that depended on labels or on in-sample τ. Evaluate all 105 defective images instead of 13 so that per-defect statements have support.

- **Controlled scoring ablation:** `hybrid` vs. `relative` vs. `mahalanobis` under the same seed and split, reported per official defect class, to quantify the full-width blind spot.
- **Speed:** compute each LBP map once per strip and histogram it per window, instead of once per overlapping patch (expected about 2× fewer LBP evaluations at stride 128), and apply the $\sigma = 50$ background blur on a downsampled strip.
- **Localization:** the nested 64×64 refinement inside flagged 256×256 patches, aimed at the 0.36 % pixel IoU.
- **Defect-free evaluation:** add the held-out defect-free strips to the test set to report image-level false alarms, which the current defective-only test set cannot measure.
- Open a pull request for `refactor/production-grade` and have the team review it before merging into `main`.

Conclusion

Week 4 turned the research scripts into a maintainable package: installable, configured through one validated config file, driven by a CLI, logged, tested and checked by CI. In doing so it uncovered and fixed problems that affected earlier results: all 13 mid-semester defect labels were wrong, the strip orientation was swapped, missing masks and missing data were replaced silently, undefined metrics were averaged as zeros, and the threshold was calibrated in-sample. Overall mid-semester figures stand, but two recall means change once undefined cases are excluded, and all per-defect analysis must be redone. Week 5 will re-run the AFID benchmark on the corrected code, with full provenance, and use the new configurability for the scoring and refinement experiments.

References

[1] J. Silvestre-Blanes, T. Albero-Albero, I. Miralles, R. Pérez-Llorens, and J. Moreno, "A public fabric database for defect detection methods and results," *AUTEX Res. J.*, vol. 19, no. 4, pp. 363–374, Dec. 2019, doi: 10.2478/aut-2019-0035.

[2] P. J. Rousseeuw and C. Croux, "Alternatives to the median absolute deviation," *J. Amer. Statist. Assoc.*, vol. 88, no. 424, pp. 1273–1283, 1993, doi: 10.1080/01621459.1993.10476408.

[3] C. Leys, C. Ley, O. Klein, P. Bernard, and L. Licata, "Detecting outliers: Do not use standard deviation around the mean, use absolute deviation around the median," *J. Exp. Social Psychol.*, vol. 49, no. 4, pp. 764–766, 2013, doi: 10.1016/j.jesp.2013.03.013.

[4] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning*, 2nd ed. New York, NY, USA: Springer, 2009, ch. 7.

[5] P. C. Mahalanobis, "On the generalized distance in statistics," *Proc. Nat. Inst. Sci. India*, vol. 2, no. 1, pp. 49–55, 1936.

[6] A. N. Tikhonov, "Solution of incorrectly formulated problems and the regularization method," *Soviet Math. Dokl.*, vol. 4, pp. 1035–1038, 1963.

[7] D. Sculley *et al.*, "Hidden technical debt in machine learning systems," in *Adv. Neural Inf. Process. Syst. 28*, 2015, pp. 2503–2511.

[8] G. Wilson *et al.*, "Best practices for scientific computing," *PLoS Biol.*, vol. 12, no. 1, e1001745, 2014, doi: 10.1371/journal.pbio.1001745.

[9] J. Pineau *et al.*, "Improving reproducibility in machine learning research," *J. Mach. Learn. Res.*, vol. 22, no. 164, pp. 1–20, 2021.

[10] R. M. Haralick, K. Shanmugam, and I. Dinstein, "Textural features for image classification," *IEEE Trans. Syst., Man, Cybern.*, vol. SMC-3, no. 6, pp. 610–621, Nov. 1973, doi: 10.1109/TSMC.1973.4309314.

[11] T. Ojala, M. Pietikäinen, and T. Mäenpää, "Multiresolution gray-scale and rotation invariant texture classification with local binary patterns," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 24, no. 7, pp. 971–987, Jul. 2002, doi: 10.1109/TPAMI.2002.1017623.

[12] K. Zuiderveld, "Contrast limited adaptive histogram equalization," in *Graphics Gems IV*, P. S. Heckbert, Ed. San Diego, CA, USA: Academic Press, 1994, pp. 474–485.